

Gift / Points Sharing Module — MySQL Documentation

Database: dev_myunisavedb

Purpose: Allow a user (the Owner) to share part of their wallet balance with other users (the Recipients) through requests, approvals, and automated distribution based on percentage allocations.

1. Core Entities

The main tables involved in the Gift / Points Sharing module are Users, Wallets, Share Requests, Share Allocations, and Transfers.

1.1 User Wallets

Stores the balance for each user.

```
CREATE TABLE IF NOT EXISTS user_wallets (  
  wallet_id BIGINT PRIMARY KEY NOT NULL,  
  user_id BIGINT,  
  balance_rs DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

1.2 Share Requests

Tracks incoming requests from recipients to an owner.

```
CREATE TABLE IF NOT EXISTS share_requests (  
  request_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  owner_id BIGINT NOT NULL,  
  requester_id BIGINT NOT NULL,  
  requested_percent DECIMAL(5,2) NOT NULL,  
  status ENUM('pending', 'accepted', 'rejected', 'cancelled') NOT NULL DEFAULT 'pending',  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  responded_at TIMESTAMP NULL DEFAULT NULL,  
  CONSTRAINT chk_req_percent CHECK (requested_percent >= 0 AND requested_percent <= 100)  
);
```

1.3 Share Allocations

Stores accepted allocations for recipients.

```
CREATE TABLE IF NOT EXISTS share_allocations (  
  allocation_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  owner_id BIGINT NOT NULL,  
  recipient_id BIGINT NOT NULL,  
  percent_share DECIMAL(5,2) NOT NULL,  
  source_request_id BIGINT NULL,  
  is_active TINYINT(1) NOT NULL DEFAULT 1,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_alloc_src FOREIGN KEY (source_request_id) REFERENCES share_requests(request_id),  
  CONSTRAINT chk_alloc_percent CHECK (percent_share >= 0 AND percent_share <= 100),  
  CONSTRAINT ux_owner_rec UNIQUE (owner_id, recipient_id)
```

```
);
```

1.4 Transfers Ledger

Tracks every distribution event.

```
CREATE TABLE IF NOT EXISTS transfers (  
  transfer_id      BIGINT PRIMARY KEY AUTO_INCREMENT,  
  owner_id        BIGINT NOT NULL,  
  recipient_id    BIGINT NOT NULL,  
  amount_rs       DECIMAL(12,2) NOT NULL,  
  percent_snapshot DECIMAL(5,2) NOT NULL,  
  created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  note            VARCHAR(255) NULL,  
  CONSTRAINT chk_tr_amt CHECK (amount_rs >= 0)  
);
```

2. Function: fn_total_percent_for_owner

Returns the total percent allocated by an owner.

```
CREATE FUNCTION fn_total_percent_for_owner(p_owner_id BIGINT)  
RETURNS DECIMAL(6,2)  
DETERMINISTIC  
READS SQL DATA  
BEGIN  
  DECLARE v_total DECIMAL(6,2);  
  SELECT IFNULL(SUM(percent_share), 0.00)  
    INTO v_total  
  FROM share_allocations  
 WHERE owner_id = p_owner_id AND is_active = 1;  
  RETURN v_total;  
END;
```

3.1 Procedure: sp_send_share_request

Sends a share request from a recipient to an owner.

```
CALL sp_send_share_request(<owner_id>, <requester_id>, <percent>);
```

```
CREATE PROCEDURE sp_send_share_request(  
  IN p_owner_id BIGINT,  
  IN p_requester_id BIGINT,  
  IN p_requested_percent DECIMAL(5,2)  
) BEGIN  
  -- validations and insert/update logic  
END;
```

3.2 Procedure: sp_respond_share_request

Owner responds to a share request (accept/reject).

```
CALL sp_respond_share_request(<request_id>, <owner_id>, TRUE|FALSE);
```

```
CREATE PROCEDURE sp_respond_share_request(  
  IN p_request_id BIGINT,  
  IN p_owner_id BIGINT,  
  IN p_accept BOOLEAN  
) BEGIN
```

```
-- logic to accept or reject requests, enforcing ≤100%
END;
```

3.3 Procedure: sp_share_owner_amount

Distributes a specified amount from the owner's wallet to recipients based on allocations.

```
CALL sp_share_owner_amount(<owner_id>, <amount_rs>, <note>);
```

```
CREATE PROCEDURE sp_share_owner_amount(
  IN p_owner_id BIGINT,
  IN p_amount_rs DECIMAL(12,2),
  IN p_note VARCHAR(255)
) BEGIN
  -- deduct, distribute using cursor, insert into transfers
END;
```

4. Example Workflow

Demonstrates usage of requests, acceptance, and sharing.

```
-- U1 asks for 10%
CALL sp_send_share_request(20, 21, 10.00);
-- U2 asks for 20%
CALL sp_send_share_request(20, 22, 20.00);
-- U3 asks for 20%
CALL sp_send_share_request(20, 23, 20.00);

-- Owner accepts requests
CALL sp_respond_share_request(1, 20, TRUE);
CALL sp_respond_share_request(2, 20, TRUE);
CALL sp_respond_share_request(3, 20, TRUE);

-- Check allocation total
SELECT fn_total_percent_for_owner(20);

-- Share Rs.200
CALL sp_share_owner_amount(20, 200.00, 'Monthly share');

-- Verify results
SELECT * FROM user_wallets WHERE user_id IN (20,21,22,23);
SELECT * FROM transfers WHERE owner_id = 20;
```

5. Notes & Extensions

- Total allocations capped at 100%. - Rounding residue returned to owner. - Revoking allocation = set is_active=0. - Ledger provides full audit trail. - Works for multiple owners independently.